Language Technology

Chapter 4: Encoding and Annotation Schemes

https://link.springer.com/chapter/10.1007/978-3-031-57549-5_4

Pierre Nugues

Pierre.Nugues@cs.lth.se

September 3, 2026



Character Sets

Codes are used to represent characters.
ASCII defines 128 code points (0 to 127) and was designed primarily for English.
Latin-1 extends this set to 256 code points. It covers most Western European languages but omits several characters, such as the French *Œ/œ*, the German quotation mark „ and the Dutch *IJ/ij*.
Latin-1 was not adopted uniformly by all operating systems (for example, classic Mac OS used a different encoding, and Windows used a variant).
Latin-9 (published in 1999) is a more complete character set for Western European languages.



Unicode

Unicode is an attempt to represent most of the world's writing systems.
From *Programming Perl* by Larry Wall, Tom Christiansen and Jon Orwant (O'Reilly, 2000):

> *If you don't know yet what Unicode is, you will soon—even if you skip reading this chapter—because working with Unicode is becoming a necessity.*

It originally used 16 bits and now uses up to 32 bits.
Code points range from 0 to 10FFFF in hexadecimal.
Unicode is the standard character representation in many operating systems and programming languages, including Java.
Each character has a code point and a name, for example:

```
U+0042 LATIN CAPITAL LETTER B
U+0391 GREEK CAPITAL LETTER ALPHA
U+00C5 LATIN CAPITAL LETTER A WITH RING ABOVE
```



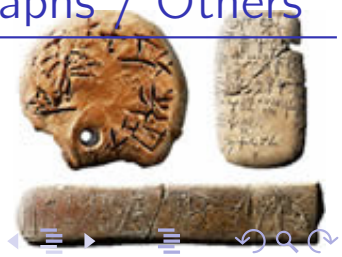
Unicode Blocks (Simplified)

Code	Name	Code	Name
U+0000	Basic Latin	U+1400	Unified Canadian Aboriginal Syllab
U+0080	Latin-1 Supplement	U+1680	Ogham, Runic
U+0100	Latin Extended-A	U+1780	Khmer
U+0180	Latin Extended-B	U+1800	Mongolian
U+0250	IPA Extensions	U+1E00	Latin Extended Additional
U+02B0	Spacing Modifier Letters	U+1F00	Greek Extended
U+0300	Combining Diacritical Marks	U+2000	General Punctuation / Symbols
U+0370	Greek and Coptic	U+2800	Braille Patterns
U+0400	Cyrillic	U+2E80	CJK Radicals Supplement
U+0530	Armenian	U+2F00	Kangxi Radicals
U+0590	Hebrew	U+3000	CJK Symbols and Punctuation
U+0600	Arabic	U+3040	Hiragana, Katakana
U+0700	Syriac	U+3100	Bopomofo
U+0780	Thaana	U+3130	Hangul Compatibility Jamo



Unicode Blocks (Simplified) (II)

Code	Name	Code	Name
U+0900	Devanagari, Bengali	U+3190	Kanbun
U+0A00	Gurmukhi, Gujarati	U+31A0	Bopomofo Extended
U+0B00	Oriya, Tamil	U+3200	Enclosed CJK Letters and Months
U+0C00	Telugu, Kannada	U+3300	CJK Compatibility
U+0D00	Malayalam, Sinhala	U+3400	CJK Unified Ideographs Extension A
U+0E00	Thai, Lao	U+4E00	CJK Unified Ideographs
U+0F00	Tibetan	U+A000	Yi Syllables
U+1000	Myanmar	U+A490	Yi Radicals
U+10A0	Georgian	U+AC00	Hangul Syllables
U+1100	Hangul Jamo	U+D800	Surrogates
U+1200	Ethiopic	U+E000	Private Use Area
U+13A0	Cherokee	U+F900	CJK Compatibility Ideographs / Others



Unicode and Python

The code point of a character and the character corresponding to a code point can be obtained with the functions `ord()` and `chr()`, respectively:

```
ord('C'), ord('Γ')    # (67, 915)
hex(67), hex(915)     # ('0x43', '0x393')
chr(67), chr(915)     # ('C', 'Γ')
```



Character Composition and Normalization

Unicode allows accented characters to be composed from a base character and one or more diacritics (e.g. $\hat{E}$ or $\mathring{A}$).
A single code point:

```
U+00CA LATIN CAPITAL LETTER E WITH CIRCUMFLEX
U+00C5 LATIN CAPITAL LETTER A WITH RING ABOVE
```

The same characters can also be represented as sequences of two code points (E + ˆ and A + ˚):

```
U+0045 LATIN CAPITAL LETTER E
U+0302 COMBINING CIRCUMFLEX ACCENT
```

and

```
U+0041 LATIN CAPITAL LETTER A
U+030A COMBINING RING ABOVE
```



Normalization

Unicode defines Normalization Form Decomposition (NFD) and Normalization Form Composition (NFC).
NFD decomposes Ê into U+0045 + U+0302 (E and combining circumflex):

```
[hex(ord(cp)) for cp in unicodedata.normalize('NFD', 'Ê')]
# ['0x45', '0x302']
```

NFC decomposes and then recomposes the character:

```
[hex(ord(cp)) for cp in unicodedata.normalize('NFC', 'Ê')]
# ['0xca']
```

In addition, Unicode defines Normalization Form Compatibility Decomposition (NFKD) and Composition (NFKC).
NFKD decomposes the *fi* ligature (U+FB01) into U+0066 + U+0069, making it equivalent to the sequence of the two letters *f i*.



Regular Expressions and Unicode Classes

Individual code points can be matched with:

- `\N{LATIN CAPITAL LETTER E WITH CIRCUMFLEX}` and `\x{CA}` (both match Ê)
- `\N{GREEK CAPITAL LETTER GAMMA}` and `\x{393}` (both match Γ)

Sets of characters can be matched with the property construct `\p{property}` or its complement `\P{property}`.
Examples:

- `\p{L}` matches letters; `\P{L}` matches non-letters
- `\p{InGreek_and_Coptic}` matches characters in a Unicode block
- `\p{Currency_Symbol}` matches currency symbols
- `\p{Greek}` matches Greek characters



Regular Expressions and Unicode Classes

Always prefer Unicode properties to legacy character classes.
To match a sequence of letters (a word), use:

`\p{L}+`

rather than

`\w+`

which is not fully standardized across languages and engines.
Python and Java may produce different results with `\w`.



General Unicode Category

Major classes		Subclasses		Major classes		Subclasses	
Short	Long	Short	Long	Short	Long	Short	Long
L	Letter			Z	Separator		
		Lu	Uppercase_Letter			Zs	Space_Separator
		Ll	Lowercase_Letter			Zl	Line_Separator
		Lt	Titlecase_Letter			Zp	Paragraph_Separator
		Lm	Modifier_Letter				
		Lo	Other_Letter				
M	Mark						
		Mn	Nonspacing_Mark				
		Mc	Spacing_Mark				
		Me	Enclosing_Mark				
N	Number						
		Nd	Decimal_Number				
		Nl	Letter_Number				
		No	Other_Number				
P	Punctuation			C	Other		
		Pc	Connector_Punctuation			Cc	Control
		Pd	Dash_Punctuation			Cf	Format
		Ps	Open_Punctuation			Cs	Surrogate
		Pe	Close_Punctuation			Co	Private_Use
		Pi	Initial_Punctuation			Cn	Unassigned
		Pf	Final_Punctuation				
		Po	Other_Punctuation				
S	Symbol						
		Sm	Math_Symbol				
		Sc	Currency_Symbol				
		Sk	Modifier_Symbol				
		So	Other_Symbol				



Legacy Regular Expressions and Unicode Compatibility

Unicode defines character classes with the constructs `\p{class}` (matches characters in the class) and `\P{class}` (matches characters not in the class).

Expression	Description	Equivalent	\p{...} equiv.
\d	Any digit	[0-9]	\p{digit}
\D	Any nondigit	[^0-9]	\P{digit}
\s	Any whitespace character (space, tab, newline, carriage return, form feed)	[\t\n\r\f]	\p{space}
\S	Any non-whitespace character	[^\s]	\P{space}
\w	Any word character (letter, digit or underscore)	[a-zA-Z0-9_]	
\W	Any non-word character	[^\w]	

Prefer the `\p{}` notation whenever possible (with the possible exception of `\s`).

See `https://www.unicode.org/reports/tr18/#Compatibility_Properties` for details.



Python and Unicode

The following instructions match lines consisting, respectively, of ASCII characters, of characters in the Greek and Coptic block, and of Greek characters:

```
import regex as re
alphabet = 'αβγδεζηθικλμνξοπρστυφχψω'
match = re.search(r'^\p{InBasic_Latin}+$', alphabet)
print(match)  # None
match = re.search(r'^\p{InGreek_and_Coptic}+$', alphabet)
print(match)  # matches alphabet
match = re.search(r'^\p{Greek}+$', alphabet)
print(match)  # matches alphabet
match = re.search(r'\N{GREEK SMALL LETTER ALPHA}', alphabet)
print(match)  # matches 'α'
match = re.search('α', alphabet)
print(match)  # matches 'α'
```



The Unicode Encoding Schemes

Unicode defines three main encoding schemes: UTF-8, UTF-16 and UTF-32.
UTF-16 was originally the default encoding scheme.
It uses fixed 16-bit units (2 bytes).
Example: *FÊTE* is encoded as `0046 00CA 0054 0045`.
UTF-8 is a variable-length encoding.
It maps the ASCII characters U+0000–U+007F to the single bytes 0x00–0x7F.
All other characters in the range U+0080–U+10FFFF are encoded as sequences of two or more bytes.



UTF-8

Range	Encoding
U+0000 – U+007F	0xxxxxxx
U+0080 – U+07FF	110xxxxx 10xxxxxx
U+0800 – U+FFFF	1110xxxx 10xxxxxx 10xxxxxx
U+10000 – U+10FFFF	11110xxx 10xxxxxx 10xxxxxx 10xxxxxx



Encoding FÊTE in UTF-8

The letters F, T and E lie in the range U+0000–U+007F.
Ê is U+00CA and lies in the range U+0080–U+07FF.
Its binary representation is 0000 0000 1100 1010.
UTF-8 uses the eleven rightmost bits of 00CA.
The first five bits together with the prefix 110 form the byte 1100 0011 (C3 in hexadecimal).
The next six bits together with the prefix 10 form the byte 1000 1010 (8A in hexadecimal).
Consequently Ê is encoded as `C3 8A` in UTF-8.
The whole string FÊTE (code points U+0046 U+00CA U+0054 U+0045) is therefore encoded as `46 C3 8A 54 45`.



Locales and Word Order

Depending on the language (and locale), dates, numbers and times are represented differently:
Numbers: 3.14 or 3,14?
Dates: 01/02/03 may mean

- 3 februari 2001 (Swedish)
- January 2, 2003
- 1 February 2003

Collation (sorting) also varies: is Andersson ordered before or after Åkesson?



The Unicode Collation Algorithm

The Unicode Consortium has defined a collation algorithm that takes into account the different practices and cultural conventions of lexical ordering.
For Latin scripts it distinguishes three levels:

- The primary level considers differences between base characters (e.g. A versus B).
- If there is no difference at the primary level, the secondary level considers diacritics (accents).
- Finally, the tertiary level considers case differences.



Differences

These level distinctions are general but not universal.
Accents are a secondary difference in many languages, yet Swedish treats accented letters as distinct base characters and therefore assigns a primary difference between A and Å, or between O and Ö.

1. Primary level: {a, A, á, Á, à, À, . . . } < {b, B} < {c, C, ć, Ć, ĉ, Ĉ, ç, Ç, . . . } < {e, E, é, É, è, È, ê, Ê, ë, Ë, . . . } < . . .
2. Secondary level: {e, E} << {é, É} << {è, È} << {ê, Ê} << {ë, Ë}
3. Tertiary level: {a} <<< {A}

At the secondary level, comparison proceeds from left to right in English, but from right to left in French.



Sorting Words in French and English

English	French
Péché	*pèche*
PÉCHÉ	*pêche*
pèche	*Pêche*
pêche	*Péché*
Pêche	*PÉCHÉ*
pêché	*pêché*
Pêché	*Pêché*
pécher	*pécher*
pêcher	*pêcher*



Markup Languages

Markup languages are used to annotate texts with structure and presentation information.
Annotation schemes used by word processors include LaTeX, RTF, etc.
XML, which resembles HTML, has become a standard annotation and exchange format.
XML is a coding framework: a language for defining ways of structuring documents.
XML is also used to represent tabulated (database-compatible) data.



XML

XML uses plain text rather than binary codes.
It separates structural instructions from content (the data).
Structure is described in a Document Type Definition (DTD) that models a class of XML documents.
A DTD contains the specific tag set used to mark up texts.
It lists the legal tags and their relationships to other tags.
XML APIs are available in many programming languages (Java, Perl, SWI-Prolog, etc.).



XML Elements

A DTD is composed of three kinds of components: elements, attributes and entities.
Elements are the logical units of an XML document.
A DocBook-like description (http://www.docbook.org/):

```
<!-- My first XML document -->
<book>
  <title>Language Processing Cookbook</title>
  <author>Pierre Cagné</author>
  <!-- The image to show on the cover -->
  <img></img>
  <text>Here comes the text!</text>
</book>
```



Differences with HTML

XML tags must be balanced: every start tag must be followed by a corresponding end tag.
Empty elements such as `<img></img>` may be written in the abbreviated form `<img/>`.
XML tags are case-sensitive: `<TITLE>` and `<title>` define different elements.
An XML document has a single root element that encloses the whole document (here `<book>`).



XML Attributes

An element may have attributes, i.e. a set of properties.
A <title> element may, for example, have an alignment (left, right or centre) and a character style (underlined, bold or italics).
Attributes appear as name–value pairs inside the start tag:

```
<title align="center" style="bold">
  Language Processing Cookbook
</title>
```



Entities

Entities are named pieces of data stored somewhere on a computer.
They may represent accented characters, symbols, strings, or even whole text or image files.
An entity reference consists of the entity name enclosed by the delimiters `&` and `;` (e.g. `&EntityName;`).
The entity reference is replaced by the corresponding entity content when the document is processed.
Useful entities include the predefined entities and the character entities.



Entities (II)

XML recognizes five predefined entities.
They correspond to characters that have a special meaning in XML and therefore cannot be written literally in a document.

Symbol	Entity encoding	Meaning
<	<	Less than
>	>	Greater than
&	&	Ampersand
"	"	Quotation mark
’	'	Apostrophe

A character reference is the Unicode code point of a single character, written either in decimal (Ê for Ê) or in hexadecimal (Ê).



Writing a DTD: Elements

A DTD specifies the formal structure of a class of documents.
A DTD file contains the declarations of all legal elements, attributes and entities.
Element declarations are enclosed between the delimiters <!ELEMENT and >:

```
<!ELEMENT book (title, (author | editor)?, img, chapter+)>
<!ELEMENT title (#PCDATA)>
```

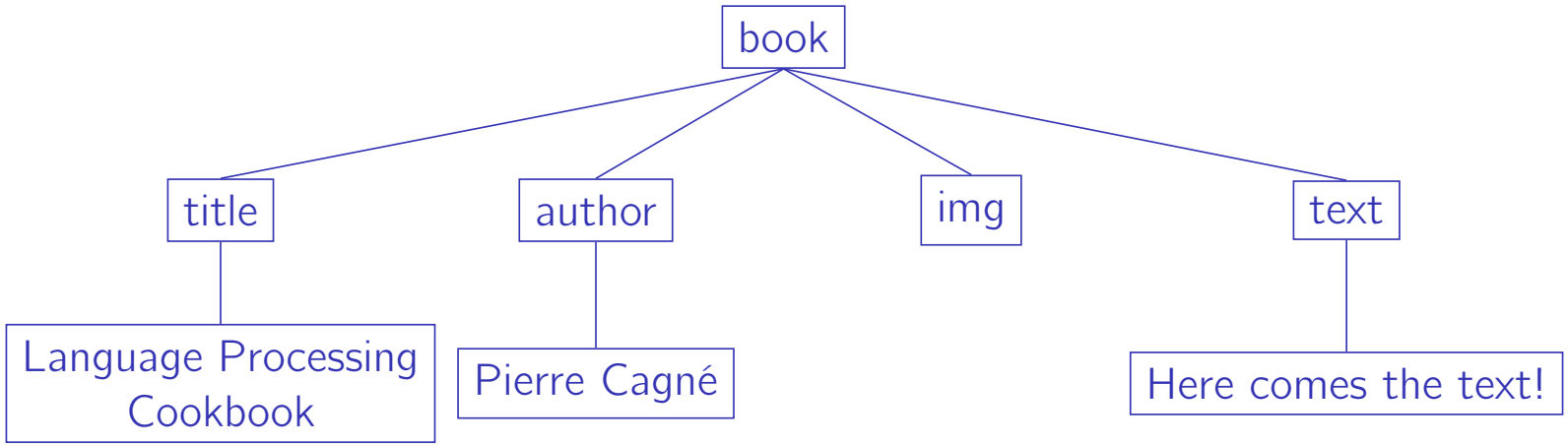


Writing an XML Document

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE book [
<!ELEMENT book (title, (author | editor)?, img, chapter+)>
<!ELEMENT title (#PCDATA)>
...
]>
<book>
  <title style="i">Language Processing Cookbook</title>
  <author style="b">Pierre Cagné</author>
  <img src="pierre.jpg"/>
  <chapter number="c1">
    <subtitle>Introduction</subtitle>
    <para>Let&apos;s start doing simple things: collect texts.
    </para>
    <para>First, choose a site you like</para>
  </chapter>
</book>
```



Tree Representation





Collecting HTML Documents

Wikipedia is a popular source of documents.
We can download the HTML content of a page from its URL, for example `https://en.wikipedia.org/wiki/Aristotle`.
The `requests` module makes it easy to build a simple scraper.
Wikimedia requires that programs identify themselves in the request header.
We can do this with a name and an e-mail address:

```
headers = {
    'User-Agent': 'PNLP/1.0 (pierre.nugues@cs.lth.se)'
}
```



Parsing HTML

We collect and parse the page with BeautifulSoup:

```
import bs4
import requests
url_en = 'https://en.wikipedia.org/wiki/Aristotle'
html_doc = requests.get(url_en, headers=headers).text
parse_tree = bs4.BeautifulSoup(html_doc, 'html.parser')
```

The variable `parse_tree` now contains the parsed HTML document; we can access its elements and their attributes.



Parsing HTML (II)

The title element and its textual content are obtained via the `title` attribute:

```
parse_tree.title
# <title>Aristotle - Wikipedia, the free encyclopedia</title>
parse_tree.title.text
# Aristotle - Wikipedia, the free encyclopedia
```



Parsing HTML (III)

The first `h1` heading corresponds to the title of the article:

```
parse_tree.h1.text
# Aristotle
```

The `h2` headings contain the section titles. We obtain the list of subtitles with the method `find_all()`:

```
headings = parse_tree.find_all('h2')
[heading.text for heading in headings]
# ['Contents', 'Life', 'Thought', 'Loss and preservation of
#  his works', 'Legacy', 'List of works', 'Eponyms', 'See also
#  'Notes and references', 'Further reading', 'External links
#  'Navigation menu']
```

